

Hidden Voices: Exploring Gender Disguise and Literary Parallels in Historical Authorship

A Machine Learning Approach to Classifying Author Gender from Biographical and Inferred Data

Israel Davidson

Table of contents

1. Introduction and hypothesis	1
Introduction	1
Hypothesis	2
2. Data and methods	2
Field definitions and distributional stats	3
Male Guess Data Summary	6
Male Biological Data Summary	7
Female Biological Data Summary	8
Combination of datasets	9
3. Results	10
Vectorizer	10
Truncated SVD	11
Guess	11
Guess	12
UMAP	12
Biological	12
Guess	13
Classification	15
4. Discussion and conclusions	22
SVD	22
UMAP	22
Classification Model	22
Limitations	23
Conclusion	23

1. Introduction and hypothesis

Introduction

Throughout history, the roles and abilities of women have often been underestimated, leading to societal restrictions that prevented them from fully participating in many fields. Women's

contributions were frequently dismissed, and many were persecuted for engaging in activities considered outside the scope of traditional female roles. For example, the mere act of a woman being literate—knowing how to read, write, and perform basic mathematical operations—was once deemed suspicious and could even lead to accusations of witchcraft. This societal limitation contributed to a phenomenon where many women had to disguise their true identities in order to access opportunities reserved for men.

Throughout history, there are many stories of women who assumed male personas to participate in male-dominated fields. Frances Clayton, for instance, was an American woman who disguised herself as a man to serve in the Union Army during the Civil War. Similarly, Mary Ann Evans, one of the most influential English novelists of the 19th century, wrote under the male pseudonym George Eliot. These instances reflect the lengths to which women had to go to gain recognition and the ability to pursue their talents in areas traditionally reserved for men.

Hypothesis

Given the historical context where women often had to conceal their identities to participate in various professions and causes, it is hypothesized that books written by women will exhibit similarities to books written by men whose gender is only inferred from their names. This assumption is based on the belief that, despite the challenges faced by women, their writing styles and content would align with the broader literary trends of their time, as women writers often adopted male pen names to avoid bias or limited opportunities.

2. Data and methods

```
import os
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import PCA, TruncatedSVD
from sklearn.preprocessing import Normalizer
from umap import UMAP
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
from sklearn.cluster import KMeans
from imblearn.over_sampling import SMOTE
from sklearn.preprocessing import LabelEncoder

from sklearn.model_selection import cross_validate
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import MultinomialNB
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV, cross_validate
```

```
from sklearn.metrics import accuracy_score,
f1_score, roc_auc_score, classification_report

import altair as alt
```

```
c:\Users\lizzyd\OneDrive\Documents\School\Cornell\Women-in-
Writing\Women\.venv\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IProgress
not found. Please update jupyter and ipywidgets. See https://ipywidgets.
readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

```
# Load the metadata
metadata = pd.read_csv(
    'corpus_data.tsv',
    sep='\t',
    low_memory=False
).set_index('source_id')
```

```
# Glance length of data
print(len(metadata))
```

```
1540
```

```
metadata=metadata[['gender', 'gender_guess', 'occupation']]
```

Field definitions and distributional stats

Most of the metadata fields are self-explanatory, but here are some details. Note that not every field in the metadata is described below.

- `source_id`: This is the name of the file corresponding to the volume. You can use it to match metadata records to full-text documents. Note that the corpus includes a nontrivial number of multivolume works. These volumes have `source_ids` like `eaf086v1` or `Wright2-1720v2`.
- `gender`: Author gender. M, F, or NaN (= unknown).
- `gender_guess`: Was the author gender assignment determined by biographical research (0) or by guessing on the basis of the author's name (1)?
- `occupation`: The author's primary employment identification. Recall that the US in the nineteenth century didn't always have a large market for novels, so many of the authors in the corpus made their living by other means.

```
metadata.head()
```

	gender	gender_guess	occupation
source_id			
eaf001	M	0.0	Arts
eaf002	F	0.0	Education
eaf003	F	0.0	Education
eaf004	F	0.0	Education
eaf005	M	0.0	Church

```
# Occurrence counts for selected metadata fields
for col in ['gender', 'gender_guess', 'occupation']:
    display(metadata[col].value_counts())
    print()
```

```
gender
M    1005
F     420
Name: count, dtype: int64
```

```
gender_guess
0.0    1144
1.0     281
Name: count, dtype: int64
```

```
occupation
Editor-Critic-Journalist    217
Writer                      187
Education                   147
Politics-Government-Activism 136
Church                      121
Military-Seafaring          72
Law                         57
Business-Trade              33
Medicine                    28
Arts                        6
Agriculture-Land            6
Name: count, dtype: int64
```

```

#Merging all of the text files corpuses into a singular dataframe
folder_path = 'corpus'

# Create an empty list to store file data
data = []

# Loop over all files in the folder
for filename in os.listdir(folder_path):
    file_path = os.path.join(folder_path, filename)

    # Ensure we're only reading files (and not folders)
    if os.path.isfile(file_path):
        with open(file_path, 'r', encoding='utf-8') as file:
            content = file.read().replace('\n', ' ') # Remove newline
            characters
            # Append a dictionary with the filename as 'id' and cleaned
            content as 'text'
            data.append({'source_id': filename, 'text': content})

# Create a DataFrame from the list of dictionaries
df = pd.DataFrame(data)

df.head()

```

	source_id	text
0	eaf001	MONALDI. CHAPTER I. Among the students of a se...
1	eaf002	THE REGICIDES We dig no lands for tyrants but ...
2	eaf003	The loved, the hated, the adored, Each mortal,...
3	eaf004	2 THE BRIDE OF FORT EDWARD. PART FIRST. INDUCT...
4	eaf005	LETTER I. Original State of the Forest .The Ad...

```

#merging both the metadata and dataframe containing our text
merged_data=metadata.merge(df,on="source_id",how="left")
merged_data.head()

```

	source_id	gender	gender_guess	occupation	text
0	eaf001	M	0.0	Arts	MONALDI. CHAPTER I. Among the students of a se...
1	eaf002	F	0.0	Education	THE REGICIDES We dig no lands for tyrants but ...

	source_id	gender	gender_guess	occupation	text
2	eaf003	F	0.0	Education	The loved, the hated, the adored, Each mortal,...
3	eaf004	F	0.0	Education	2 THE BRIDE OF FORT EDWARD. PART FIRST. INDUCT...
4	eaf005	M	0.0	Church	LETTER I. Original State of the Forest .The Ad...

Male Guess Data Summary

```
male_guess=merged_data[(merged_data['gender']=='M') &
(merged_data['gender_guess']==1.0)]
print(len(male_guess))
male_guess.head()
```

160

	source_id	gender	gender_guess	occupation	text
122	eaf085	M	1.0	NaN	B LETTERS OF MAJOR J. DOWN-ING LETTER I. FROM T...
129	eaf091v1	M	1.0	NaN	2 TRAVELING. MENTALLY AND BODILY. It is a whol...
130	eaf091v2	M	1.0	NaN	VOL. II. 2 STREETS OF LONDON. In few places ar...
451	eaf394	M	1.0	NaN	GREAT ATTRACTION! OR THE DOCTOR MOST OUDACIOUS...
452	eaf395	M	1.0	NaN	MAJOR JONES'S SKETCHES OF TRAVEL Through the U...

```
# Occurrence counts for selected metadata fields
display(male_guess['occupation'].value_counts(dropna=False))
```

```
occupation
NaN    160
Name: count, dtype: int64
```

I find it interesting that the entire dataframe for when the author is assigned Male only based off of their name that they all had an occupation of NaN

Male Biological Data Summary

```
male_bio=merged_data[(merged_data['gender']=='M') &
(merged_data['gender_guess']==0.0)]
print(len(male_bio))
male_bio.head()
```

845

	source_id	gender	gender_guess	occupation	text
0	eaf001	M	0.0	Arts	MONALDI. CHAPTER I. Among the students of a se...
4	eaf005	M	0.0	Church	LETTER I. Original State of the Forest .The Ad...
5	eaf006	M	0.0	Writer	PART I. CHAPTER I. INTRODUCTIONTHE TRAVELERS. ...
6	eaf007	M	0.0	Writer	CHAPTER I. Change is written on the tide On th...
7	eaf008	M	0.0	Writer	CHAPTER I. INTRODUCTIONTHE STRANGERTHE ATTACKT...

```
# Occurrence counts for selected metadata fields
display(male_bio['occupation'].value_counts(dropna=False))
```

```
occupation
Editor-Critic-Journalist    180
Church                      120
NaN                         100
Politics-Government-Activism  97
Writer                      84
Education                   76
Military-Seafaring          72
Law                         57
Medicine                    25
Business-Trade              22
Arts                        6
Agriculture-Land            6
Name: count, dtype: int64
```

Female Biological Data Summary

```
fe_bio=merged_data[(merged_data['gender']=='F') &
(merged_data['gender_guess']==0.0)]
print(len(fe_bio))
fe_bio.head()
```

299

	source_id	gender	gender_guess	occupation	text
1	eaf002	F	0.0	Education	THE REGICIDES We dig no lands for tyrants but ...
2	eaf003	F	0.0	Education	The loved, the hated, the adored, Each mortal,...
3	eaf004	F	0.0	Education	2 THE BRIDE OF FORT EDWARD. PART FIRST. INDUCT...
32	eaf026	F	0.0	Writer	RECITAL. THE FIRE-SIDE. Various misfortunes had...
52	eaf041	F	0.0	Politics-Government-Activism	1* HOBOMOK. CHAP. I. How daur ye try sic sport...

```
# Occurrence counts for selected metadata fields
display(fe_bio['occupation'].value_counts(dropna=False))
```

```
occupation
Writer          103
Education       71
Politics-Government-Activism  39
Editor-Critic-Journalist    37
NaN             34
Business-Trade    11
Medicine          3
Church            1
Name: count, dtype: int64
```


Combination of datasets

Now I will be creating two separate dataframes to perform my tests on. I will be combining the researched biological male and female datasets as well as combining the biological male dataset and the guessed one

```
bio=pd.concat([male_bio,fe_bio],axis=0).set_index('source_id')
bio.head()
```

	gender	gender_guess	occupation	text
source_id				
eaf001	M	0.0	Arts	MONALDI. CHAPTER I. Among the students of a se...
eaf005	M	0.0	Church	LETTER I. Original State of the Forest .The Ad...
eaf006	M	0.0	Writer	PART I. CHAPTER I. INTRODUCTION-THE TRAVELERS. ...
eaf007	M	0.0	Writer	CHAPTER I. Change is written on the tide On th...
eaf008	M	0.0	Writer	CHAPTER I. INTRODUCTIONTHE STRANGER THE ATTACKT...

```
guess=pd.concat([male_bio,male_guess],axis=0).set_index('source_id')
guess['gender_guess_label'] = guess['gender_guess'].map({0.0: "Biological",
1.0: "Guessed"})
guess.head()
```

	gender	gender_guess	occupation	text	gender_guess_label
source_id					
eaf001	M	0.0	Arts	MONALDI. CHAP- TER I. Among the students of a se...	Biological
eaf005	M	0.0	Church	LETTER I. Original State of the For- est .The Ad...	Biological
eaf006	M	0.0	Writer	PART I. CHAP- TER I. INTRODUC- TIONTHE TRAV- ELERS. ...	Biological

source_id	gender	gender_guess	occupation	text	gender_guess_label
eaf007	M	0.0	Writer	CHAPTER Change is written on the tide On th...	I. Biological
eaf008	M	0.0	Writer	CHAPTER INTRODUC- TIONTHE STRANGER THE ATTACKT...	I. Biological

3. Results

Vectorizer

For this block we shall set up our text vectorizer that will be used on both of our dataframes

```
vectorizer = TfidfVectorizer(
    strip_accents='unicode', # Normalize and remove accents from characters
    lowercase=True,          # Convert all text to lowercase to ensure
    consistency
    use_idf=True,             # Apply Inverse Document Frequency (IDF)
    weighting to emphasize rare words
    min_df=0.05,              # Ignore terms that appear in less than 5% of
    documents (helps remove rare noise)
    max_df=0.9,               # Ignore terms that appear in more than 90% of
    documents (removes overly common words)
    ngram_range=(1, 2),       # Extract unigrams (single words) to 2-grams
    (phrases up to 2 words)
    stop_words='english'      # Remove common English stop words (e.g.,
    'the', 'and', 'is')
    ,max_features=47354
)

# Vectorize and normalize the text data
features_guess = vectorizer.fit_transform(guess['text'])
features_bio = vectorizer.fit_transform(bio['text'])
print("Feature matrix shape of bio dataframe:", features_bio.shape)
print("Feature matrix shape of guess dataframe:", features_guess.shape)
```

```
Feature matrix shape of bio dataframe: (1144, 47354)
Feature matrix shape of guess dataframe: (1005, 47354)
```

Truncated SVD

Guess

```
X_reduced_bio =  
TruncatedSVD(random_state=42,n_components=2).fit_transform(features_bio)  
  
svd_bio=pd.DataFrame({'SVD1':X_reduced_bio[:,0], 'SVD2':X_reduced_bio[:,1], 'Gender':bio['gender'],  
svd_bio.head(10)
```

	SVD1	SVD2	Gender	Story
source_id				
eaf001	0.422303	-0.184385	M	MONALDI. CHAPTER I. Among the students of a se...
eaf005	0.224659	-0.021976	M	LETTER I. Original State of the Forest .The Ad...
eaf006	0.250992	-0.007070	M	PART I. CHAPTER I. INTRODUCTIONTHE TRAVELERS. ...
eaf007	0.231117	-0.048055	M	CHAPTER I. Change is written on the tide On th...
eaf008	0.383394	-0.098360	M	CHAPTER I. INTRODUCTIONTHE STRANGERTHE ATTACKT...
eaf009	0.342446	-0.109134	M	THE TRAPPER'S BRIDE CHAPTER I. O'er vales that...
eaf011	0.209957	0.024813	M	OLIVER GOLDFINCH. CHAPTER I. THE HYP-OCRITE UNM...
eaf012	0.349550	-0.099551	M	BOOK I. CHAPTER I. WE have seen that a sort of...
eaf013v1	0.365774	-0.462372	M	CHAPTER I. In the year of Grace fifteen hun-dre...
eaf013v2	0.341024	-0.481506	M	CHAPTER I. The day that followed after the fli...

```
alt.Chart(svd_bio,title=f"Scatterplot of Truncated SVD for biologically  
defined Males and Females").mark_circle(size=60).encode(  
  x='SVD1',  
  y='SVD2',  
  color='Gender',  
  tooltip=['Gender', 'Story']  
) .properties(  
  width=700,
```

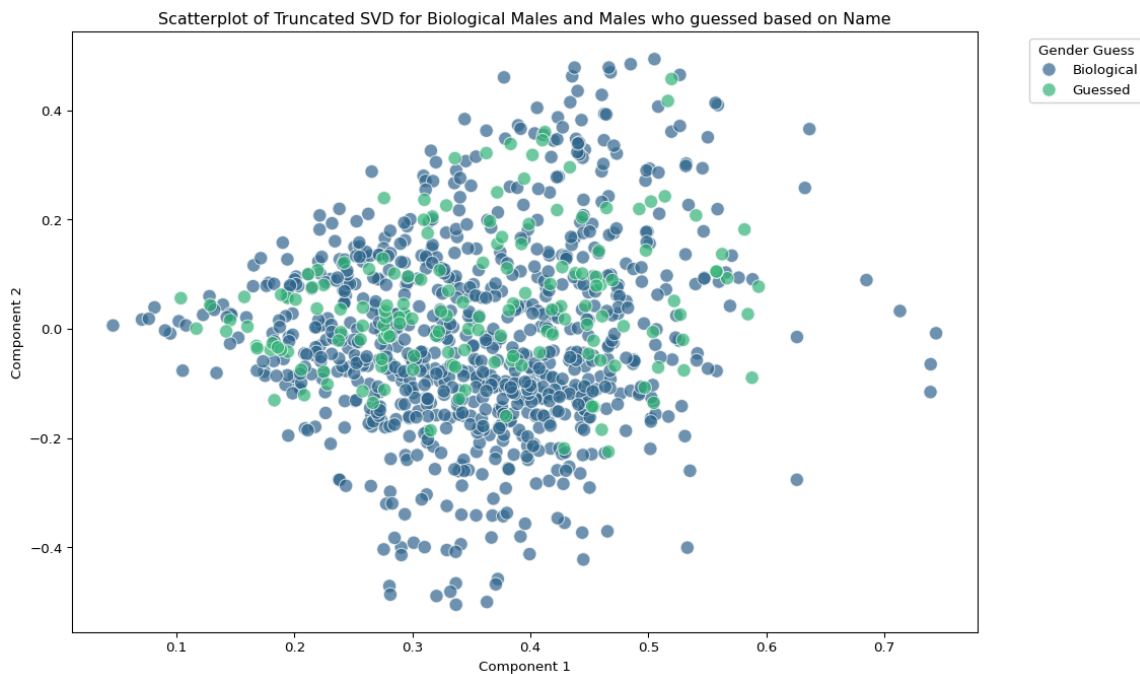
```
height=500
).interactive()
```

```
alt.Chart(...)
```

Guess

```
X_reduced_guess =
TruncatedSVD(random_state=42,n_components=2).fit_transform(features_guess)
```

```
plt.figure(figsize=(12, 8))
sns.scatterplot(x=X_reduced_guess[:,0], y=X_reduced_guess[:,1],
hue=guess['gender_guess_label'],alpha=0.7,s=100,palette='viridis')
plt.title("Scatterplot of Truncated SVD for Biological Males and Males who
guessed based on Name")
plt.xlabel("Component 1")
plt.ylabel("Component 2")
plt.legend(title="Gender Guess", bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()
```



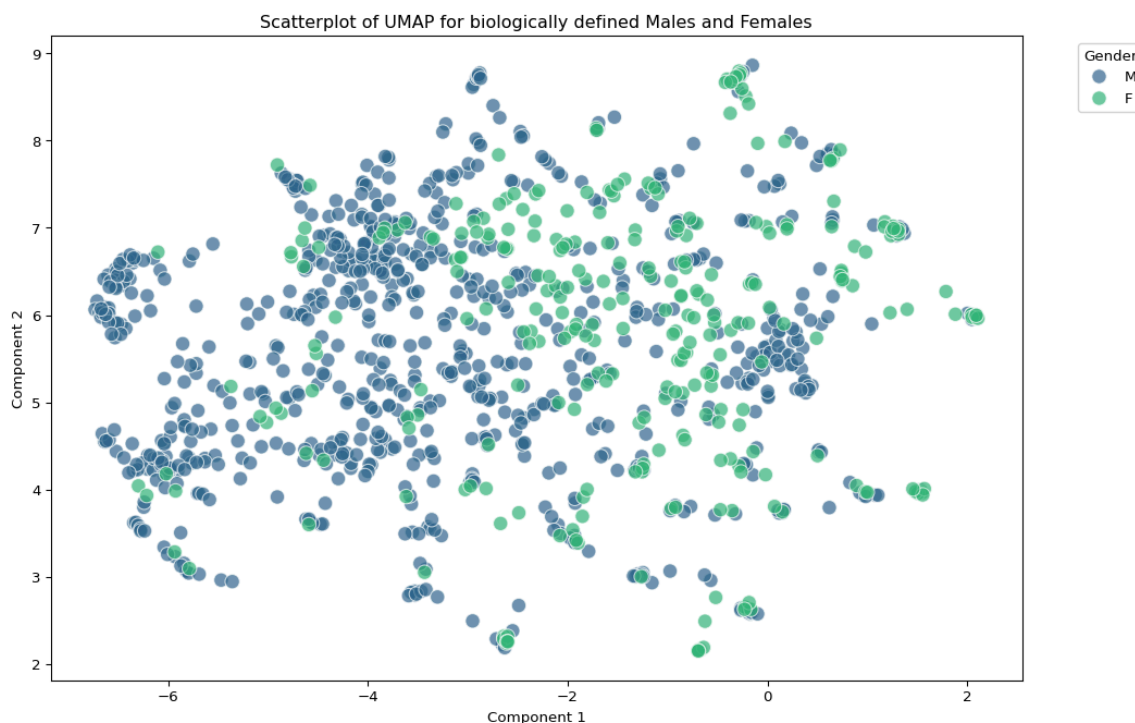
UMAP

Biological

```
X_umap_bio = UMAP(random_state=42,n_components=2).fit_transform(features_bio)
```

```
c:\Users\izzyd\OneDrive\Documents\School\Cornell\Women-in-
Writing\Women\.venv\Lib\site-packages\sklearn\utils\deprecation.py:151:
FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6
and will be removed in 1.8.
  warnings.warn(
c:\Users\izzyd\OneDrive\Documents\School\Cornell\Women-in-
Writing\Women\.venv\Lib\site-packages\umap\umap_.py:1952: UserWarning: n_jobs
value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
  warn(
```

```
plt.figure(figsize=(12, 8))
sns.scatterplot(x=X_umap_bio[:,0], y=X_umap_bio[:,1],
hue=bio['gender'],alpha=0.7,s=100,palette='viridis')
plt.title("Scatterplot of UMAP for biologically defined Males and Females")
plt.xlabel("Component 1")
plt.ylabel("Component 2")
plt.legend(title="Gender", bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()
```

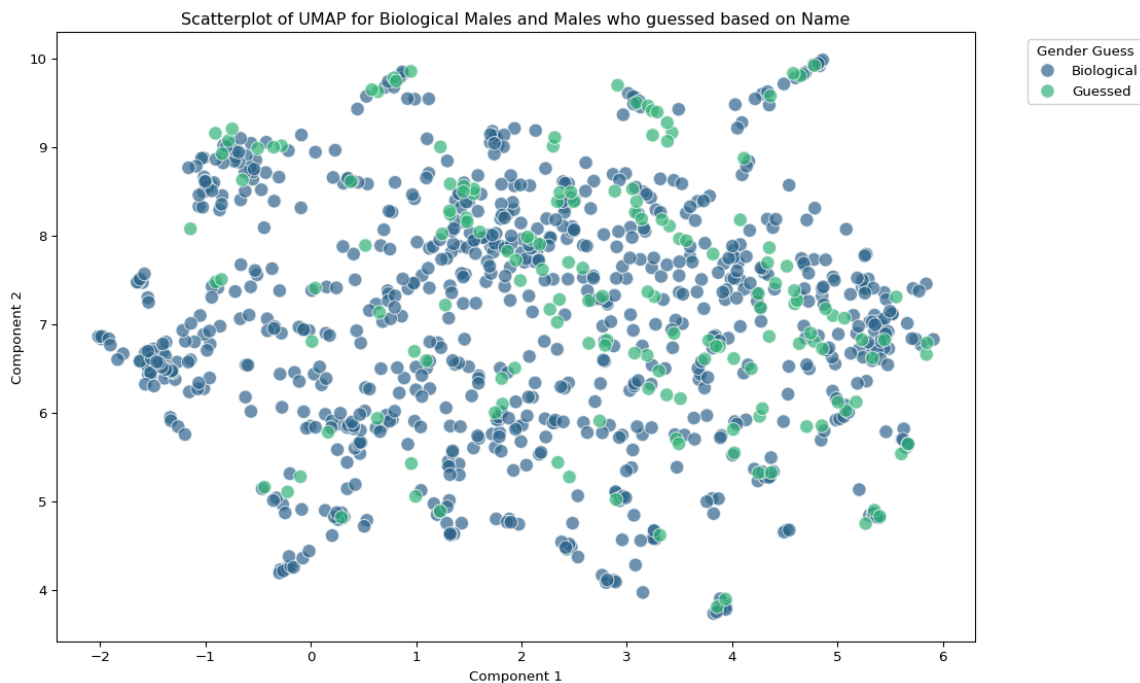


Guess

```
X_umap_guess =
UMAP(random_state=42,n_components=2).fit_transform(features_guess)
```

```
c:\Users\lizzyd\OneDrive\Documents\School\Cornell\Women-in-
Writing\Women\.venv\Lib\site-packages\sklearn\utils\deprecation.py:151:
FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6
and will be removed in 1.8.
  warnings.warn(
c:\Users\lizzyd\OneDrive\Documents\School\Cornell\Women-in-
Writing\Women\.venv\Lib\site-packages\umap\umap_.py:1952: UserWarning: n_jobs
value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
  warn(
```

```
plt.figure(figsize=(12, 8))
sns.scatterplot(x=X_umap_guess[:,0], y=X_umap_guess[:,1],
hue=guess['gender_guess_label'],alpha=0.7,s=100,palette='viridis')
plt.title("Scatterplot of UMAP for Biological Males and Males who guessed
based on Name")
plt.xlabel("Component 1")
plt.ylabel("Component 2")
plt.legend(title="Gender Guess", bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()
```



Classification

For this I will be creating different models to be able to predict the gender of an author. The idea here is that I will be taking the best model and will check to see if the model will mostly guess if the what the gender of an author is if the gender guess column is 1. For this step I will be using both of the datasets to complete this experiment. My biological dataset will be used in order to train the model and the male guess model to perform predictions on it. To somewhat deal with the imbalance of data points in both datasets I will be using the SMOTE technique as we already used a vectorizer on most of the data

```
def compare_scores(scores_dict):  
    """  
    Takes a dictionary of cross_validate scores.  
    Returns a color-coded Pandas dataframe that summarizes those scores.  
    """  
    df =  
    pd.DataFrame(scores_dict).T.map(np.mean).style.background_gradient(cmap='RdYlGn')  
    return df
```

```
# Initialize LabelEncoder  
encoder_bio = LabelEncoder()  
  
# Fit the encoder separately on the 'gender' column of the bio dataset  
encoder_bio.fit(['M', 'F'])  
y_bio_encoded = encoder_bio.transform(bio['gender']) # 'M' becomes 1, 'F'  
becomes 0
```

I am using SMOTE on the data to fix the imbalance of the male and female classes in the bio dataset, why creating synthetic examples

```
#SMOTE Defining  
  
#Bio  
smote = SMOTE(random_state=42)  
X_smote_bio, y_smote_bio = smote.fit_resample(features_bio, y_bio_encoded)  
X_smote_bio_df = pd.DataFrame(X_smote_bio.toarray()) # Convert sparse matrix  
to dense
```

```
#baseline  
# Calculate baseline performance  
# Determine the most common label  
most_common_label = bio['gender'].mode()[0] # Get the most common label  
  
# Create predictions (assign most common label to every observation)  
predictions = [most_common_label] * len(bio)
```

```

# Calculate baseline accuracy
baseline_accuracy = accuracy_score(bio['gender'], predictions)

# Calculate baseline F1 score
baseline_f1 = f1_score(bio['gender'], predictions, average='weighted')

# Print the results
print(f"Baseline Accuracy: {baseline_accuracy:.4f}")
print(f"Baseline F1 Score: {baseline_f1:.4f}")

# Score metrics to be used
scoring = ['accuracy', 'precision', 'recall', 'f1', 'roc_auc']

```

Baseline Accuracy: 0.7386
Baseline F1 Score: 0.6276

```

%%time
# Classifiers to test
classifiers = {
    'kNN': KNeighborsClassifier(),
    'Logit': LogisticRegression(max_iter=1000),
    'Decision Tree': DecisionTreeClassifier(),
    'Random Forest': RandomForestClassifier(),
    'Multinomial NB': MultinomialNB()
}

scores_dict = {}
for clf_name, clf in classifiers.items():
    cv_results = cross_validate(clf, X_smote_bio, y_smote_bio, cv=5,
                                scoring=scoring)
    scores_dict[clf_name] = cv_results
# Compare scores with the function
compare_scores(scores_dict)

```

CPU times: total: 3min 35s
Wall time: 3min 37s

	fit_time	score_time	test_accuracy	test_precision	test_recall	test_f1	test_roc_auc
kNN	0.220065	13.115492	0.566272	0.963165	0.137278	0.238284	0.682476
Logit	0.788939	0.026818	0.908876	0.955700	0.861538	0.901103	0.962144
Decision Tree	19.104788	0.371390	0.826627	0.865175	0.811834	0.825773	0.826627

	fit_time	score_time	test_accuracy	test_precision	test_recall	test_f1	test_roc_auc
Random Forest	8.081840	1.510582	0.910059	0.921057	0.921893	0.914156	0.988512
Multinomial NB	0.148827	0.041259	0.844379	0.959457	0.719527	0.814620	0.952061

```
%%time

# List of hyperparameters to tune
n_estimators_range = [100, 200, 1000]
max_depths_range = [5, 10, 15]

# Placeholder for storing scores
rf_scores_dict = {}

# Loop through combinations of n_estimators and max_depth
for n_estimators in n_estimators_range:
    for max_depth in max_depths_range:
        # Initialize RandomForestClassifier with current hyperparameters
        rf = RandomForestClassifier(n_estimators=n_estimators,
                                   max_depth=max_depth, random_state=42)

        # Perform cross-validation
        cv_results = cross_validate(rf, X_smote_bio, y_smote_bio, cv=5,
                                    scoring=scoring) # Replace 'accuracy' with your preferred scoring metric

        # Store results
        rf_scores_dict[(n_estimators, max_depth)] = cv_results

# Display the results (assuming compare_scores is a function to display the results)
compare_scores(rf_scores_dict)
```

CPU times: total: 22min 37s
Wall time: 22min 50s

		fit_time	score_time	test_accuracy	test_precision	test_recall	test_f1	test_roc_auc
100	5	4.168086	1.554359	0.897633	0.907749	0.914793	0.903305	0.981492
	10	6.928158	1.487737	0.910059	0.920064	0.924260	0.914472	0.988971
	15	7.909719	1.516853	0.908284	0.918459	0.923077	0.913228	0.989531
200	5	7.365447	2.591564	0.897633	0.909472	0.914793	0.903577	0.983768
	10	13.225125	2.650253	0.909467	0.922409	0.919527	0.913347	0.989888

		fit_time	score_time	test_accuracy	test_precision	test_recall	test_f1	test_roc_auc
	15	15.108415	2.625382	0.913018	0.923232	0.925444	0.917158	0.990340
1000	5	34.587360	11.701044	0.905325	0.915661	0.923077	0.911045	0.984188
	10	63.801459	11.673086	0.908876	0.921784	0.919527	0.912897	0.990364
	15	73.386833	11.710500	0.912426	0.921784	0.926627	0.916978	0.990624

We see from the tables above that a Random Forest Model with 15 n_estimators and a max_depth of 10000 is the best resulting model. For this next part of my classification I will be fitting a model with our SMOTE biological data. We do this to train the model on the entire dataset. However we shall be making my predictions using the entire dataset were the author's gender is assigned as M but has a mixture of if it is assigned because of research or because of guessing the author's name

```
#Since My Random Forest did the best we shall use it to make our predictions
# Initialize Random Forest classifier
rf_classifier =
RandomForestClassifier(n_estimators=15,max_depth=1000,random_state=42)

# Train the model using Bio data
rf_classifier.fit(X_smote_bio_df, y_smote_bio)

# Predict on Guess data
y_pred_guess = rf_classifier.predict(features_guess)
```

```
predictions_df = pd.DataFrame({
    'Predicted': y_pred_guess,
    'Guess_Label':guess['gender_guess_label'],
    'Gender':guess['gender'],
    "Guess":guess['gender_guess']
})

predictions_df.head()
```

	Predicted	Guess_Label	Gender	Guess
source_id				
eaf001	1	Biological	M	0.0
eaf005	1	Biological	M	0.0
eaf006	1	Biological	M	0.0
eaf007	1	Biological	M	0.0
eaf008	1	Biological	M	0.0

Remember from our label encoder that M is as 1 and F is marked as 0. Also remember that Guess is saved as 1 while research that backs up the gender of the author is saved as 0. For this I will be checking the errors within the predictions to see what kinds of inference can be pulled with this information.

```
# Entries where Guess is 1 but Predicted is 1
guess_1_predicted_1 = predictions_df[(predictions_df['Guess'] == 1.0) &
(predictions_df['Predicted'] == 1)]
print("The Model predicting Male when the author gender is guessed")
print(len(guess_1_predicted_1))
guess_1_predicted_1
```

The Model predicting Male when the author gender is guessed
147

	Predicted	Guess_Label	Gender	Guess
source_id				
eaf085	1	Guessed	M	1.0
eaf091v1	1	Guessed	M	1.0
eaf091v2	1	Guessed	M	1.0
eaf394	1	Guessed	M	1.0
eaf395	1	Guessed	M	1.0
...	...	...	...	...
Wright2-2480	1	Guessed	M	1.0
Wright2-2483	1	Guessed	M	1.0
Wright2-2484	1	Guessed	M	1.0
Wright2-2487	1	Guessed	M	1.0
Wright2-2490	1	Guessed	M	1.0

```
# Entries where Guess is 1 but Predicted is 0
guess_1_predicted_0 = predictions_df[(predictions_df['Guess'] == 1) &
(predictions_df['Predicted'] == 0)]
print("The Model predicting Female when the author gender is guessed")
print(len(guess_1_predicted_0))
guess_1_predicted_0
```

The Model predicting Female when the author gender is guessed
13

	Predicted	Guess_Label	Gender	Guess
source_id				
Wright2-0010	0	Guessed	M	1.0
Wright2-0014	0	Guessed	M	1.0
Wright2-0016	0	Guessed	M	1.0
Wright2-0296	0	Guessed	M	1.0
Wright2-0328	0	Guessed	M	1.0
Wright2-0385	0	Guessed	M	1.0
Wright2-0388	0	Guessed	M	1.0
Wright2-0474	0	Guessed	M	1.0
Wright2-0533	0	Guessed	M	1.0
Wright2-0539	0	Guessed	M	1.0
Wright2-0631	0	Guessed	M	1.0
Wright2-0638	0	Guessed	M	1.0
Wright2-1655	0	Guessed	M	1.0

```
# Entries where Guess is 1 but Predicted is 0
guess_0_predicted_0 = predictions_df[(predictions_df['Guess'] == 0) &
(predictions_df['Predicted'] == 0)]
print("The Model predicting Female when the author gender is not guessed")
print(len(guess_0_predicted_0))
guess_0_predicted_0
```

The Model predicting Female when the author gender is not guessed
66

	Predicted	Guess_Label	Gender	Guess
source_id				
eaf015v2	0	Biological	M	0.0
eaf017v1	0	Biological	M	0.0
eaf018v2	0	Biological	M	0.0
eaf019v1	0	Biological	M	0.0
eaf025v1	0	Biological	M	0.0
...	...	...	...	...

	Predicted	Guess_Label	Gender	Guess
source_id				
Wright2-1702	0	Biological	M	0.0
Wright2-1721	0	Biological	M	0.0
Wright2-1740	0	Biological	M	0.0
Wright2-1785	0	Biological	M	0.0
Wright2-2278	0	Biological	M	0.0

```
guess_0_predicted_1 = predictions_df[(predictions_df['Guess'] == 0) &
(predictions_df['Predicted'] == 1)]
print("The Model predicting Male when the author gender is not guessed")
print(len(guess_0_predicted_1))
guess_0_predicted_1
```

The Model predicting Male when the author gender is not guessed
779

	Predicted	Guess_Label	Gender	Guess
source_id				
eaf001	1	Biological	M	0.0
eaf005	1	Biological	M	0.0
eaf006	1	Biological	M	0.0
eaf007	1	Biological	M	0.0
eaf008	1	Biological	M	0.0
...	...	...	...	...
Wright2-2750	1	Biological	M	0.0
Wright2-2751	1	Biological	M	0.0
Wright2-2752	1	Biological	M	0.0
Wright2-2801	1	Biological	M	0.0
Wright2-2802	1	Biological	M	0.0

4. Discussion and conclusions

SVD

When we first analyzed the graphs generated by SVD on the biological gender of the author, we observed that male authors did not exhibit a distinct pattern on the plot. They were spread across the graph without a clear clustering or trend. In contrast, female authors, while also somewhat spread out, tended to stay within a similar region, forming a more noticeable cluster. This clustering suggests that there may be some common characteristics shared by female authors that set them apart from male authors. However, the spread of male authors across the plot suggests a lack of a strong, inherent pattern for gender-specific writing traits, at least in terms of the features analyzed by SVD.

When analyzing the data where the gender of the author was guessed, the pattern remained similar to the biological gender data. This suggests that the guess about the author's gender, whether accurate or not, did not drastically change the overall distribution of authors in the plot. The lack of a distinct gender-based pattern in either biological or guessed gender data further supports the idea that there may not be strong, easily detectable differences in the writing styles based solely on gender.

UMAP

The UMAP plots revealed a subtle distinction in the biological gender data, with male and female authors showing slight separation. However, this difference was minimal and not pronounced enough to draw definitive conclusions. For the guessed gender data, the UMAP plots showed no discernible patterns, further reinforcing the idea that writing style is not inherently linked to gender, even when gender is assigned based on names rather than biological information. These findings challenge the assumption that gender significantly influences writing style, at least based on the features used in this analysis.

Classification Model

To explore the potential connection between an author's gender and their writing style, a classification model was trained using data from authors whose biological gender was known. The model was then applied to the guessed gender data to assess its accuracy. The objective was to determine whether the model would struggle with guessed gender data, particularly when predicting male authors. A consistent misclassification of guessed gender data would suggest a lack of discernible patterns in writing style related to gender.

Interestingly, the model achieved a high ROC value of 0.99, indicating exceptional performance with minimal errors. Most guessed male authors were accurately classified as male by the model, suggesting that many guessed male authors were indeed biologically male. While historical context highlights instances where women used male pseudonyms to publish, this finding supports the likelihood that the majority of guessed male authors were accurately classified. This further suggests that gendered writing styles, if they exist, may not be strongly pronounced in the features analyzed.

Limitations

- **Gender Classification by the Researcher:** The gender of many authors in the dataset was inferred by the researcher based on limited biographical information, such as names. This subjective classification introduces potential bias, which may undermine the reliability of the hypothesis and the model's predictions. Misclassifications or assumptions about an author's gender could skew the results and influence the observed patterns.
- **Lack of Genre Information:** The absence of genre data posed another challenge. Different genres often adhere to distinct writing conventions that may influence an author's style. Historically, books were not categorized by genre as they are today, making it difficult to control for genre-related variations in the analysis. This limitation may have affected both the visualizations produced by SVD and UMAP and the performance of the classification model. Including genre information could provide deeper insights into whether observed patterns are more reflective of genre conventions than gender-specific traits.

Conclusion

This study aimed to explore whether there is a discernible difference in writing styles between male and female authors, based on the assumption that gender influences literary expression. The hypothesis posited that books written by women would exhibit similarities to those written by men whose gender is inferred from their names, due to historical instances of women adopting male personas to access male-dominated fields.

However, the findings from the SVD and UMAP analyses indicate that gender does not significantly influence writing style. The results showed only subtle differences between male and female authors, with no clear, distinct patterns emerging in either the biological or guessed gender data. The lack of a strong trend suggests that gender does not have a major impact on the literary characteristics of authorship, contrary to the hypothesis.

Furthermore, the classification model, while highly accurate in predicting the gender of guessed authors, suggests that the guessed gender data still likely aligns with biological gender, particularly for male authors. This reinforces the idea that the guessed gender data did not significantly differ from the biological gender data in terms of writing style.

Given these findings, the hypothesis that gender influences writing style and that books written by women would closely resemble those written by men (as inferred from their names) is rejected. The study provides evidence that gender does not play a decisive role in shaping literary style, at least not in a way that can be easily detected through the methods and features analyzed in this research. Future studies with more detailed data and further examination of other factors influencing writing style, such as genre, may offer more insight into the complex relationship between an author's identity and their literary work.